

COMP2001: ARTIFICIAL INTELLIGENCE

METHODS – LAB 6 EXERCISES

RECOMMENDED TIMESCALES

This lab exercise is designed to take no longer than two hours, and it is recommended that you complete this within a week. The concepts of this exercise were covered in lecture 6.

GETTING SUPPORT

The computing exercises have been designed to facilitate flexible studying and can be worked through in your own time or during timetabled lab hours. It is recommended that you attend the labs, even if you have completed the exercises in your own time, to discuss your solutions and understanding with one of the lab support team and/or your peers. It is entirely possible that you might make a mistake in your solution which leads to a false observation and understanding.

OBJECTIVES

The purpose of this lab exercise is to gain experience in implementing a multimeme memetic algorithm and observing how the process of co-evolving memes can adapt to the problem being solved. Multimeme memetic algorithms were covered in lecture 6.

- COMP2001 framework background for handling populations of solutions.
- Implementation of a multimeme memetic algorithm.
- Empirical understanding of the co-evolution process.

LEARNING OUTCOMES

By completing this lab exercise, you should meet the following learning outcomes:

- Students should gain experience in implementing advanced (multi point/population based) metaheuristics for solving combinatorial optimisation problems.
- Students should be able to explain the process of co-evolution and draw on conclusions of the usefulness of neighbourhood operators and various parameter settings based on empirical results.

TASK 1 – IMPORTING LAB EXERCISE FILES

Download the source code for this lab exercise from the Moodle page. You should find various files related to the multimeme memetic algorithm, the exercise runner configurations, and the exercise runners themselves. Drag the `multimeme` folder into your IDE so that it is a direct subfolder of `"com.aim.metaheuristics.population"`.

The `"Lab6ExercisesTestFrameConfig"` and `"Lab6ExercisesRunner"` classes should be placed within the `"runners"` package alongside the ones from previous labs.

TASK 2 – READING: FRAMEWORK BACKGROUND

A multi-meme memetic algorithm extends the standard memetic algorithm by introducing memeplexes to each solution in the population. Within the framework, each memeplex is referenced within each solution as illustrated below with each memeplex containing a set of memes. These memes are stored in an array and can be referenced by their meme index. In the below example meme m_1 is in meme index 0, and meme m_2 is in meme index 1.

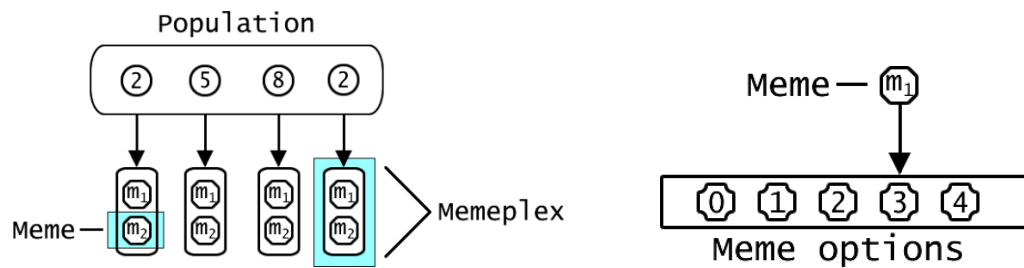


Figure 1 - Representation of the solution memory with memes and memeplexes (left) and association of memes with meme options (right).

Each meme points to a specific setting/option which symbolises its genetic code. Each individual meme has a specific number of different options, which in this case is 5. These are integer values starting at 0 and ending at the number of options **exclusive**. You may want to create a mapping of these integer values to each option's characteristic, for example local search operator.

In a multi-meme memetic algorithm, an individual is associated with a memeplex containing a set of memes. The memeplex is intrinsic to each individual and can be accessed using the `getMeme(int iSolutionIndex, int iMemeIndex)` method on the SAT Object where `iSolutionIndex` is the memory index where the associated solution along with the associated meme are stored, and `iMemeIndex` is the index of the meme in the memeplex. In the COMP2001 Framework, a meme is encoded as an integer value as shown in the table below. In the example, the memeplex contains 3 memes. A meme option is the discrete integer value that a meme is assigned to. Memes are represented by a `Meme` Object. Within this object, you can do two things; get the current meme option and set the meme option. For example, if we get the meme from solution index 0 and meme index 2 ($M_{0,2}$), then `getMemeOption()` will return 5.

Solution Memory	Genetic representation	Memeplex
[0] (parent 1)	$s_i(P_1) = 1100101101$	meme[0] = 2
		meme [1] = 0
		meme [2] = 5
[1] (parent 2)	$s_i(P_2) = 1100101101$	meme [0] = 3
		meme [1] = 1
		meme [2] = 7
[2] (child 1)	$s_i(C_1) = 1100101101$	meme [0] = 1
		meme [1] = 0
		meme [2] = 3

[3] (child 2)	$s_i(C_2) = 1100101101$	meme [0] = 5
		meme [1] = 6
		meme [2] = 9

TASK 3 – MULTIMEME MEMETIC ALGORITHMS [IMPLEMENTATION]

Multimeme Memetic Algorithms (MMA's) were covered in lecture 6. Below is the pseudocode for a Multimeme Memetic Algorithm embedding memplexes for deciding which local search operator to apply and the intensity of mutation setting to use for the mutation step.

```

1 | INPUT: PopulationSize, MaxGenerations, InnovationRate
2 | generateInitialPopulation();
3 | FOR 0 -> MaxGenerations
4 |     FOR 0 -> PopulationSize / 2
5 |         select parents using tournament selection
6 |         apply crossover to generate offspring
7 |         inherit memplex using simple inheritance method
8 |         mutate the memes within each memplex of each child with
           ↵ probability dependent on the innovation rate
9 |         apply mutation to offspring with intensity of mutation set for
           ↵ each solution dependent on its meme option
10 |        apply local search to offspring with choice of operator
           ↵ dependent on each solution's meme option
11 |     ENDFOR
12 |     do population replacement
13 | ENDFOR
14 | return  $s_{best}$ ;

```

Code 1 - Pseudocode for the Multimeme Memetic Algorithm.

Like the previous labs, you only need to implement the main loop of the Multimeme Memetic Algorithm (lines 5-10). The outermost loop and initialisation are handled by Lab6ExerciseRunner.java.

MULTIMEME MEMETIC ALGORITHM

You are given 2 Classes, MultiMeme.java and SimpleInheritanceMethod.java, which you will need to complete to implement the Multimeme Memetic Algorithm. The multimeme code reuses your implementations of crossover, mutation, local search, population replacement, and tournament selection from the previous tasks.

Within MultiMeme.java, we have specified three methods which we ask that you implement and call from runMainLoop() to perform the respective operations. This has the advantage of making your code neater and easier to implement. These methods are as follows:

- applyMutationForChildDependentOnMeme(int childIndex, int memeIndex);
- applyLocalSearchForChildDependentOnMeme(int childIndex, int operatorMemeIndex, int depthOfSearchMemeIndex);
- performMutationOfMemplex(int solutionIndex);

MUTATION BASED ON MEMES

The first meme (meme in index 0 of the memplex) will be used to represent the intensity of mutation setting. Meme 0 has 5 possible options which are mapped to intensity of mutation settings as $iom \leftarrow option$. That is, the intensity of mutation setting equals the meme option value.

The mutation operator is the same as `BitMutation` from lab 5 but whereas the `MUTATION_RATE` was set as $1/chromosome_length$ previously, the rate of mutation now depends on an intensity of mutation setting equal to $x/chromosome_length$ where x is the integer intensity of mutation setting. There is a method `setMutationRate(int intensityOfMutation)` which should be called on the mutation heuristic to set the aforementioned parameter setting.

LOCAL SEARCH BASED ON MEMES

The **second meme** (meme in index 1 of the memplex) will be used to represent the local search operator to use. There are four meme options, and these are mapped to operators as in the following map:

- map | 0 -> DBHC accepting only improving moves.
- | 1 -> DBHC accepting non-worsening moves.
- | 2 -> SDHC accepting only improving moves.
- | 3 -> SDHC accepting non-worsening moves.

Note that this is the default ordering given in the `MultiMeme.java` code (see ``private final PopulationHeuristic[] lss;``).

The **third meme** (meme in index 2 of the memplex) will be used to represent a depth of search parameter setting similar to that we used when implementing Iterated Local Search. The meme option value should be used as the number of times the local search operator is applied.

MEMEPLEX MUTATION

Mutation of the memes within each memplex should be performed based on the given ``innovationRate`` and such that it is possible to mutate the meme in memplex index 0 but not the meme in memplex index 1. In other words, the random probability to mutate is used to decide whether to mutate each meme rather than the memplex as a whole.

You should be careful that when a meme is chosen to be mutated, not to set its option value to be the same as the current option.

SIMPLE INHERITANCE METHOD

Within the simple inheritance method (SIM), all memes (the memplex) of each offspring are inherited from the best parent where the best parent is the parent with the best objective value. In the event of a tie, the parent from which the memes are inherited is chosen randomly. Below is the **pseudocode** for SIM. Information relating to the actual methods can be found both in the API document and in the sections “Framework Background” and “Implementation Hints”.

```

1 | INPUT: parent1, parent2, child1, child2
2 | IF f(parent1) == f(parent2) THEN
3 |     inherit = random  $\in \{parent1, parent2\}$ 
4 |     child1.memplex <- inherit.memplex
5 |     child2.memplex <- inherit.memplex
6 | ELSEIF f(parent1) < f(parent2) THEN
7 |     child1.memplex <- parent1.memplex
8 |     child2.memplex <- parent1.memplex
9 | ELSE
10 |     child1.memplex <- parent2.memplex
11 |     child2.memplex <- parent2.memplex
12 | ENDIF
13 | return;

```

Code 2 - Pseudocode for Simple Inheritance Method.

TASK 4 – FORMATIVE ASSESSMENT (MOODLE QUIZ)

The questions in the “Lab 6 Formative Assessment” require you to perform some experiments using your implementation of the multimeme memetic algorithm. Answer the questions by configuring the `Lab6ExercisesRunner` and `Lab6TestFrameConfig` classes as appropriate, running the algorithms, and analysing the resulting outputs.

ADDITIONAL INFORMATION AND TIPS

How do I get the current option/value of a meme?

The Meme class defines a method `getMemeOption()` which returns its current value. The Meme object itself can be extracted from the memplex of a solution by using the `getMeme(...)` method defined on the SAT problem object.

How do I set a new option/value of a meme?

The Meme class defines a method `setMemeOption(...)`. The Meme object itself can be extracted from the memplex of a solution by using the `getMeme(...)` method defined on the SAT problem object.

How do I know how many options there are per meme?

The Meme class defines a method `getTotalOptions()` which returns an int value. The valid options for the meme are hence in `[0, getTotalOptions())`.

How do I: perform a bit flip; get the number of variables; evaluate the solution; etc.

See the COMP2001 Framework API for framework specific questions on Moodle!